Applied Soft Computing Journal 164 (2024) 112032

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Inverse distance weight-assisted particle swarm optimized indoor localization

Jingxue Bi [a,b], Jianhui Wang [a], Hongji Cao [a,*], Guobiao Yao [a,*], Yunjia Wang [c], Zengke Li [c], Meng Sun [c], Hongchao Yang [c], Jie Zhen [d], Guoqiang Zheng [a]

[a] School of Surveying and Geo-Informatics, Shandong Jianzhu University, Jinan, 250101, Shandong Province, China
[b] State Key Laboratory of Satellite Navigation System and Equipment Technology, The 54th Research Institute of China Electronics Technology Group Corporation, Shijiazhuang, 050081, Hebei Province, China
[c] School of Environment and Spatial Informatics, China University of Mining and Technology, Xuzhou, 221116, Jiangsu Province, China
[d] Chinese Academy of Surveying and Mapping, Beijing, 100830, China

ARTICLE INFO

Keywords:
Indoor localization
WiFi
IDW
PSO
Online optimization

ABSTRACT

Indoor localization using wireless fidelity (WiFi) fingerprint has attracted considerable attention due to its extensive deployment and low cost. Most indoor fingerprint localization methods could be computationally inefficient and behave with unsatisfactory positioning accuracy. Therefore, an inverse distance weight (IDW) assisted particle swarm optimized (PSO) indoor localization (IDWPSOInLoc) algorithm is proposed. It is an optimized positioning process in the online stage rather than an offline optimization approach. The main idea is to find the optimal particle with the most similar generated fingerprint to the real-time testing one in the online stage. K nearest neighbors and boundary constraints are combined to initialize particles' positions. The generated fingerprint of each particle is interpolated based on the IDW algorithm and nearby reference points within a specified range. A new fitness function, the sum of standard deviations between the generated and real-time testing fingerprints, is proposed to find the optimal particle. The position of the optimal particle with the minimal fitness value is taken as the estimated coordinate through iterations updating the particles' velocities, positions, and generated fingerprints. IDWPSOInLoc achieves a mean positioning accuracy of 2.477 meters in the local environment experimental test and that of about 6 meters using selected floor in the UJIIndoorLoc dataset. Compared with state-of-the-art algorithms, positioning error is decreased by at least 11.9%. In addition, IDWPSOInLoc has the characteristic of low computational complexity. Experimental results show that the proposed IDWPSOInLoc outperforms the considered WiFi indoor positioning algorithms.

1. Introduction

Indoor localization is an important research topic in the fields of location-based service, artificial intelligence, and smart city [1,2]. With the rapid growth of wireless sensor networks and intelligent devices, the massive demand for indoor continuous and accurate location service drive the development of indoor localization. However, pedestrian moving, complex indoor topologies, severe non-line-of-sight, multipath and multifloor effects, lead to indoor localization more challenges in positioning accuracy and stability.

Many indoor positioning techniques were proposed achieving positioning accuracy from centimeter-level to 5 m based on wireless fidelity (WiFi) [3–5], Bluetooth [2,6], microphone [7,8], ultra-wideband (UWB) [9,10], inertial measurement unit (IMU) [11,12], camera [13,14], magnetic field [15,16] and pseudolite [17]. Considering investment, coverage and hardware support, it is difficult for some techniques to apply in a large area. WiFi is widely available in almost all buildings greatly reducing the installation workload and expense. Due to WiFi networks' extensive deployment and low-cost properties, WiFi localization has become one of the most ubiquitous and effective indoor positioning methods for a large area. Currently, there are three data sources used for WiFi localization, namely received signal strength indicator (RSSI) [3], channel state information (CSI) [4], and round trip time (RTT) [5]. Obtaining CSI and RTT usually needs some special transmitting or receiving devices, while RSSI can be captured by most

* Corresponding authors.
E-mail addresses: bijingxue19@sdjzu.edu.cn (J. Bi), 2022160111@stu.sdjzu.edu.cn (J. Wang), caohongji22@sdjzu.edu.cn (H. Cao), yao7837005@sdjzu.edu.cn (G. Yao), wyj4139@cumt.edu.cn (Y. Wang), zengkeli@yeah.net (Z. Li), msun@cumt.edu.cn (M. Sun), hongchao_yang@cumt.edu.cn (H. Yang), zhenjie@casm.ac.cn (J. Zhen), 11765@sdjzu.edu.cn (G. Zheng).

https://doi.org/10.1016/j.asoc.2024.112032
Received 12 April 2023; Received in revised form 12 April 2024; Accepted 20 July 2024
Available online 23 July 2024
1568-4946/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

smart devices, so RSSI-based indoor localization is one of the most suitable indoor ubiquitous positioning techniques[18,19].

RADAR [20] and Horus [21] are the two most classic RSSI-based fingerprint positioning systems. These systems are grounded in fundamental principles: K nearest neighbor (KNN) for RADAR and Bayesian theory for Horus. Some improved works are carried out based on these two systems, e.g., weighted KNN (WKNN) [22], the selection of dynamic K values [23,24], and kernel functions into probability distribution [25,26]. Meanwhile, more machine learning or deep learning methods are introduced for localization performance improvements, such as clustering [24,27,28], support vector machine (SVM) [29–31], artificial neural network (ANN) [32–34], Gaussian process (GP) [35–37], extreme learning machine (ELM) [38–40], and deep learning [41–43], etc. Clustering methods are adopted to split large area into small partitions to deal with the problem of large error and high complexity for large-scale positioning. Other advanced learning methods are employed to train models that establish the mapping relationship between signal and location in the offline stage. When the coverage of the indoor environment is small, these trained models could achieve good positioning accuracy. Nevertheless, in a large indoor area, under the influence of shelters, pedestrians, temperature and humidity, and other environmental changes, wireless signals in indoor scenarios will experience reflections, scattering, diffraction, and other phenomena during propagation. These signals will arrive at the receiving station through different paths, leading to dynamic changes in RSSI. Even if the transmitting and receiving stations remain fixed, RSSI values will vary due to the diverse indoor layouts and environment factors. Therefore, the real-time RSSI measurement often differs from the offline fingerprints.

Existing indoor localization methods usually train models in the offline stage, and then the location is predicted under the assumption of minimal divergence between the offline fingerprints and real-time RSSI measurement. These methods are not enough to cope with the real-time dynamic changes of the indoor environment. They usually have poor positioning performance or generalization ability. It is necessary to study an online optimization indoor localization method. There are few online optimization indoor localization researches. To deal with the above issue,an inverse distance weight-assisted particle swarm optimized indoor localization algorithm (IDWPSOInLoc) is proposed to reduce the impact of dynamic signal fluctuations and improve localization performance. It is an optimized process to partially eliminate the influence of dynamic signal changes in the online stage. The main contributions of the proposed algorithm can be summarized.

- IDWPSOInLoc is an optimized positioning process that utilizes the PSO algorithm to find the optimal particle (i.e., positioning results) with the most similar fingerprint in the online stage, rather than optimizing localization model parameters in the offline stage. This process could deal well with the dynamic changing wireless signals resulting in enhancing localization performance.
- The proposed fitness function is the sum of standard deviations (SSD) between the testing and generated fingerprints. IDWPSOInLoc based on SSD shows higher positioning accuracy than other familiar fitness functions.
- IDWPSOInLoc outperforms state-of-the-art positioning methods with an accuracy improvement of at least 11.9% using both the local environment and the selected UJIIndoorLoc datasets. Moreover, it behaves lower computational complexity.

The remainder of this paper is organized as follows. The related work is reviewed in Section 2. IDWPSOInLoc algorithm is detailed in Section 3. Section 4 provides experiments and performance evaluation. Section 5 discusses the term *large area*. Finally, the conclusion is drawn in Section 6.

2. Related work

There are few online optimized indoor localization methods. More attentions are paid to optimize parameters and models in the offline stage. Existing WiFi fingerprint-based indoor localization methods can be divided into two categories based on whether the method contains a trained model. The method without a trained model only needs to build the required fingerprint database in the offline stage. However, the method with a trained model takes more time and cost to construct a fingerprint database and train a model. Moreover, the method with a trained model could be treated as a continuous process, while the other method can be seen as a discrete process. Currently, the PSO algorithm is widely exploited in trained models of indoor localization for hyperparameters optimization to seek better performance, e.g., PSO-SVM, PSO-BP. Therefore, this section focuses on two kinds of fingerprint localization methods with/without trained model and applications of the PSO algorithm in fingerprint-based indoor localization.

2.1. Fingerprint localization method without trained model

These methods usually only carry on simple data processing to get mean or standard deviation values in the offline stage. Building a fingerprint database is a very difficult task, it needs to consume a lot of manpower and material resources. Fortunately, some well-known WiFi fingerprint database are available, such as UJIIndoorLoc [44], SODIndoorLoc [45], Tampere [46] and UTMInDualSymFi [47], which can be directly used without the need for modifications to the positioning program. In the online stage, these methods obtain location by selecting several RPs with the maximum similarity or the minimum distance from the fingerprint database. The classical methods are KNN, WKNN, and Bayesian estimation. The recognized positioning accuracy is from 3 to 5 m.

RADAR [20] firstly investigated fingerprint localization by using KNN. By calculating Euclidean distances between the testing fingerprint and reference fingerprints, it selected K RPs within the smallest distances and took the mean coordinates as the estimated location. WKNN was investigated to prompt positioning accuracy by obtaining weighted mean coordinates [22], and distance was commonly used to calculate weights. Some researchers focused on representing the similarity between the testing fingerprint and reference fingerprints [48], such as correlation coefficient, Manhattan distance, and Mahalanobis distance. Some focused on the effect of different K values to avoid large positioning error. Due to the complex and variational indoor environment, the fixed K value could not obtain good positioning results. Dynamic selection of trusted RPs based on RSSI differences or RPs' location agglomeration was explored to improve positioning accuracy and stability [23,24].

2.2. Fingerprint localization method based on trained model

Fingerprint localization methods based on trained models are usually computing unfriendly in the offline stage. The process of building a model requires traversing almost all reference fingerprints. Multifold cross-verification is usually adopted to prompt the accuracy of the trained model. In many cases, the default number of folds is 5. It is the reason for spending a lot of time and resources on training models. When the dimensions of fingerprints and the number of RPs are large, and the complexity of the adopted machine learning method is high, the required time and resources are immense. The trained model can directly predict location with the testing fingerprint in the online stage. The online stage consumes less time and resources than the offline stage.

Several machine learning methods, e.g., SVM, ANN, GP, ELM, CNN, DNN and RNN, are widely introduced for training models to build strong mapping relationships between fingerprints and locations. And they are mainly applied to the offline stage. Fingerprints are learned

well by these machine learning methods, resulting in different fine-trained models. And from the view of the trained model, the fingerprint in signal space uniquely corresponds to a position in coordinate space. It is the main reason for these trained models with better positioning accuracy. Many researchers described experimental scenarios and results in detail, except for the time and resources required to train the model. The mean positioning error using SVM is about 2.2 m in an area of more than 600 square m, 31 m in length, and 21 m in width [29]. When the test is conducted in 460 square m (23 m by 20 m), the mean positioning error is about 2 m using SVM [49]. ANN achieves a mean positioning accuracy of 2.3 m in an area of 45 m multiplied by 25 m [32], while the accuracy decreases to 2.5 m through ANN and cross-validation in an area of 180 square m [50]. These trained models have consistently demonstrated commendable good positioning performance in square areas, and their parameters are typically set as empirical values. Some parameter selections could prompt positioning accuracy in a large area or a long corridor by using GP [36], ELM [40], or EA-CNN [42], where the improvements are 18.7%, 44.2%, and 25%, respectively. This phenomenon suggests that parameter selection or optimization tremendously influences positioning performance.

2.3. Fingerprint localization model parameters optimization with PSO

Hyperparameters significantly affect the training precision and testing accuracy and are required to train fingerprint localization models. Each set of parameters makes the performance of the trained model different. For example, the insensitive coefficient, penalty factor, the width of the kernel, maximum number of iterations, and tolerance for optimization are common hyperparameters in the SVM algorithm, while the parameter of SVM is the support vector. The whole set of hyperparameters needs to be adjusted continuously to get appropriate support vectors. The size of hidden layers, activation function, the solver for weight optimization, penalty factor, size of batches, learning rate for weight updates, the initial learning rate, maximum number of iterations, tolerance for optimization, and other optional parameters can be treated as the hyperparameters of the ANN algorithm. They will determine the ANN model and its parameters, i.e., weights and thresholds, and affect the output. These hyperparameters are artificially set, and tuning them is a complex task. Therefore, optimization algorithms, such as grid search, generic algorithm, PSO, and their variants [32,50–52] are adopted to adjust the parameters.

PSO [53] works by having a population (called a swarm) of candidate solutions (called particles), and solves a problem by iteratively concerning a given fitness function and trying to move these particles around in the search space over the particle's position and velocity. Each particle's movement is guided by its local-best position in the search space and the entire swarm's best position. The expectation is to move the swarm towards the best solutions. PSO makes few or no assumptions about optimized problems and can search for many candidate solutions repeatedly. Also, it does not use the gradient of the problem being optimized, which means PSO does not require that the optimization problem be differentiable as is required by classic optimization methods such as gradient descent and quasi-newton methods [54,55]. Furthermore, PSO is superior to GA in many aspects [56]. Among these optimized algorithms, PSO is easier to implement and obtain a globally optimal solution with rapid convergence. Therefore, PSO is widely utilized for the parameter optimization of trained models for fingerprint localization in the offline stage. The optimal location could be estimated using these fine-trained models with optimized parameters in the online stage.

RSSI changes result in fingerprint differences between offline and online stages. Regardless of the presence of trained models, these existing fingerprint localization methods would behave with fluctuant positioning accuracy because of no global optimization in the online stage. SVM and ANN could achieve higher positioning accuracy in a square area but a poor performance in a long corridor due to more serious multipath effect and more complex dynamic changing signals [29,32,49,50]. This paper proposed an inverse distance weight-assisted particle swarm optimized indoor localization method to overcome the shortcomings of the existing fingerprint localization algorithms, achieving accurate indoor localization results in practical application with lower computational complexity. The proposed method is an online optimization positioning process that utilizes the PSO algorithm to find the optimal particle (i.e., positioning results) with the most similar fingerprint in the online stage. While the above mentioned methods only optimized localization model parameters in the offline stage. The most significant difference is that the proposed method focuses on obtaining online optimal positioning results, rather than offline optimization of model parameters.

3. IDWPSOInLoc method

Firstly, the overview of IDWPSOInLoc method is firstly introduced. Then, four pivotal components of the proposed algorithm are described in detail. Finally, the analysis of time complexity is conducted.

3.1. The overview of IDWPSOInLoc method

The structure of IDWPSOInLoc method is illustrated in Fig. 1, comprising two stages, the offline stage and the online stage. In the offline stage, after collecting RSSI measurements at each RP, a simple fingerprint operation is conducted to get the mean RSSI value. The RSSI value is assigned as a small negative value for the undetectable AP. Then, mapping relationships between locations and fingerprints are formed as a fingerprint database. During the online stage, the real-time RSSI readings are compared to RPs in the fingerprint database to calculate distances. K RPs are selected by KNN with the first K minimum Euclidean distances. The core component is marked in orange and outputs the estimated location.

Maximum and minimum coordinates of the selected RPs are utilized to generate location boundaries, constrain particles' movement, and accelerate the convergence speed. The velocity boundary is also set up with positive and negative velocities to avoid particles moving too fast. Positions of particles are randomly initialized within the location boundary. Once the position of each particle is determined, RPs within a range of a particle can be found. Subsequently, the fingerprint of the particle can be interpolated based on these RPs and the IDW algorithm. The fitness value of each particle can be calculated and expressed as the sum of standard deviations between interpolated fingerprint and testing fingerprint. Firstly, these particles are considered as their own local-best individuals. The particle with minimum fitness value is treated as the global-best individual. During iteration, the fitness values of these particles are newly calculated, and the local-best individuals and the global-best individual in history are replaced by comparing with corresponding fitness values. Under the influence of the local-best individuals and the global-best individual, particles constantly update their positions and move towards the local-best individuals and the global-best individual. The location of the global-best individual is regarded as the output of the estimated location until terminating the iteration.

3.2. Particles initialization

Initialization of each particle mainly generates the location and moving speed based on several selected RPs from KNN. In this subsection, KNN is not used to estimate location but to select nearby RPs around the particle.

Euclidean distance in signal space d_i^{sig} between the testing fingerprint $\boldsymbol{r}$ and the reference fingerprint $\boldsymbol{R}_i$ can be calculated by (1), where $\boldsymbol{r}$ denotes as $\boldsymbol{r} = \begin{bmatrix} r_1 & r_2 & \cdots & r_M \end{bmatrix}^{\mathrm{T}}$, $\boldsymbol{R}_i$ is the fingerprint of ith RP, $\boldsymbol{R}_i = \begin{bmatrix} R_{i,1} & R_{i,2} & \cdots & R_{i,M} \end{bmatrix}^{\mathrm{T}}$, r and R are RSSI values in testing

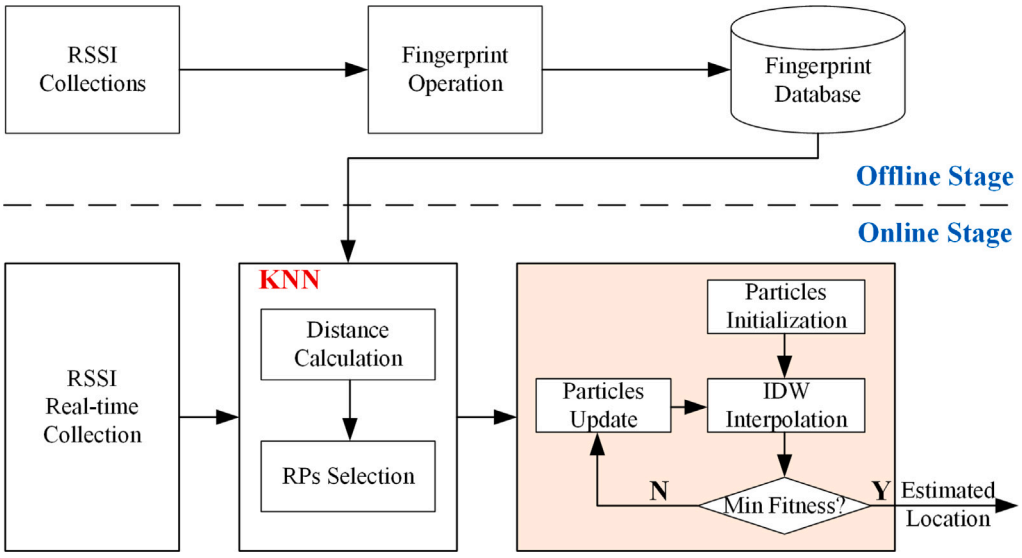


Fig. 1. The illustration of IDWPSOInLoc method.

and reference fingerprints, M is the number of media access control addresses.

$$d_i^{\text{sig}} = \sqrt{\sum_{j=1}^{M} \left(r_j - R_{i,j}\right)^2} \tag{1}$$

If there are N PRs, N Euclidean distances are obtained. These distances are sorted in ascending order, and RPs corresponding to the first K minimum distances are selected for the subsequent particle's initialization. Maximum and minimum coordinate values are generated using maximum and minimum functions in (2), where $\boldsymbol{x}$ and $\boldsymbol{y}$ denote coordinates of K selected RPs, respectively, and can be expressed as (3), x_i and y_i representing coordinate values of the ith selected RP.

$$\begin{cases} \left(x_{\max}, y_{\max}\right) = \max\left(x, y\right) \\ \left(x_{\min}, y_{\min}\right) = \min\left(x, y\right) \end{cases} \tag{2}$$

$$\left(\boldsymbol{x}, \boldsymbol{y}\right) = \begin{bmatrix} x_1 & \cdots & x_i & \cdots & x_K \\ y_1 & \cdots & y_i & \cdots & y_K \end{bmatrix}^{\mathrm{T}} \tag{3}$$

Maximum and minimum coordinate values form the location boundary to constrain the movement of particles and accelerate the convergence speed. The location boundary can be any shape, e.g., rectangle, square, circle, ellipse, and polygons. For convenience, the location boundary $\boldsymbol{B}$ is chosen as a rectangle, and represented as (4). In this case, the length and width of the location boundary can be calculated by (5).

$$\boldsymbol{B} \sim \begin{bmatrix} x_{\min} & y_{\min} \\ x_{\max} & y_{\max} \end{bmatrix} \tag{4}$$

$$\begin{cases} \mathrm{d}x = x_{\max} - x_{\min} \\ \mathrm{d}y = y_{\max} - y_{\min} \end{cases} \tag{5}$$

Then, Eq. (6) randomly generates the location of the ith particle l_i^{p} with the location boundary, where $\mathrm{rand}()$ is the random function to generate a random number from 0 to 1. And velocities of the ith particle $\boldsymbol{v}_i^{\mathrm{p}}$ can be randomly generated using (7), where $\boldsymbol{v}_i^{\mathrm{p}}$ represents a velocity vector, v_{b} is the boundary of velocity, and $\mathrm{rands}(2)$ can generate two random numbers from -1 to 1. In this way, particles are in different directions. All particles will move within the location boundary, as shown in Fig. 2.

$$\begin{cases} x_i^{\mathrm{p}} = x_{\min} + \mathrm{rand}() * \mathrm{d}x \\ y_i^{\mathrm{p}} = y_{\min} + \mathrm{rand}() * \mathrm{d}y \end{cases} \tag{6}$$

$$\boldsymbol{v}_i^{\mathrm{p}} = \left(v_{x,i}^{\mathrm{p}}, v_{y,i}^{\mathrm{p}}\right) = v_{\mathrm{b}} * \mathrm{rands}(2) \tag{7}$$

3.3. IDW interpolation for generating fingerprint of particle

IDW interpolation is used to generate the fingerprint of a particle by selecting several RPs within a specific range. The process of generating the fingerprint of a particle is illustrated in Fig. 3. Based on this process, each particle could obtain its own interpolated fingerprint.

Assuming that there are m particles, and the green particle with known coordinate is appointed in Fig. 3(a). Euclidean distance in location space between the particle and the ith RP can be calculated using (8), where $(x^{\mathrm{p}}, y^{\mathrm{p}})$ and (x_i, y_i) are the coordinates of the particle and the ith RP, respectively.

$$d_i^{\mathrm{loc}} = \sqrt{\left(x^{\mathrm{p}} - x_i\right)^2 + \left(y^{\mathrm{p}} - y_i\right)^2} \tag{8}$$

When all distances between the particle and RPs have been calculated, they are compared with a specific range. RPs within a specific range are selected, shown in Fig. 3(b), as well as their distances and fingerprints. Fingerprints of these RPs are further used for interpolating the fingerprint of the particle. The influence of reference fingerprints on the particle are determined by the inverse of distances, known as weights, as illustrated in Fig. 3(c). Given the number of RPs within a specific range of the particle is n, the weight of the ith RP on the particle can be calculated using (9). Then, the fingerprint of the particle is generated by the IDW algorithm in (10), $\boldsymbol{R}^{\mathrm{p}}$ is the generated fingerprint of the particle, $\boldsymbol{w}$ is the vector of weights from the n selected RPs, $\boldsymbol{R}_n$ is the reference fingerprints of the n selected RPs.

$$w_i = \frac{1/d_i^{\mathrm{loc}}}{\sum_{i=1}^{n} 1/d_i^{\mathrm{loc}}} \tag{9}$$

$$\boldsymbol{R}^{\mathrm{p}} = \boldsymbol{w} \cdot \boldsymbol{R}_n = \left[w_1 \cdots w_i \cdots w_n\right]^{\mathrm{T}} \begin{bmatrix} R_{1,1} & \cdots & R_{1,M} \\ \vdots & \ddots & \vdots \\ R_{n,1} & \cdots & R_{n,M} \end{bmatrix} \tag{10}$$

3.4. Fitness function

Each particle in the particle swarm has the potential to be the optimal solution with the fitness function serving as a decisive criterion. A fitness function is a particular objective function that describes how close a given solution or variable is to the objective. We propose the fitness function to be the SSD between the interpolated fingerprint and the testing fingerprint. The principle is that the more similar two fingerprints are, the smaller the SSD will be. The fitness function helps to find the fingerprint with the minimum SSD value in the signal space. The corresponding position of the particle with the minimal SSD value

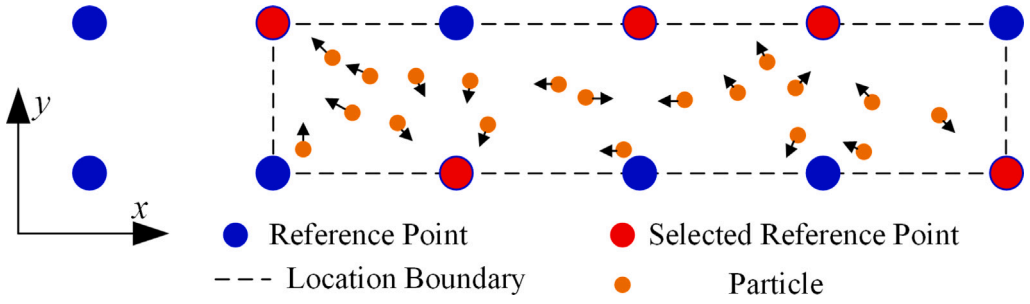


Fig. 2. Illustration of particles initialization.

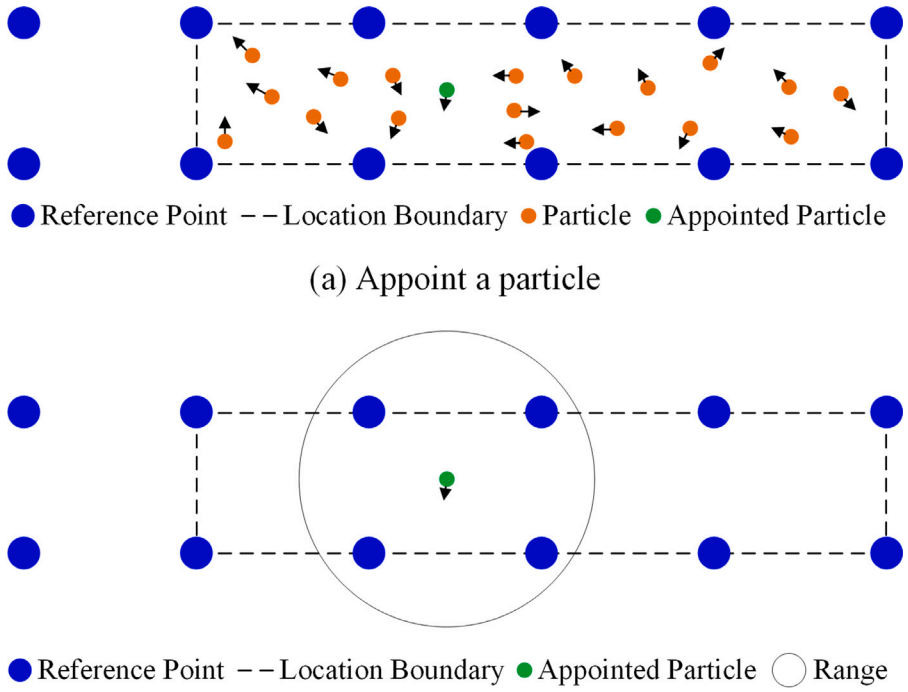


Fig. 3. The process of generating the fingerprint of a particle.

is delivered as the optimal location estimation. To our knowledge, it is the first time adopting the SSD as a fitness function.

Mean square error (MSE) is a traditional fitness function of many optimization algorithms. It is widely utilized to reflect the differences between the observed or estimated values and the actual values. The biggest difference is that two values are subtracted from the average in SSD, while one is subtracted from the other in MSE. For example, x_1 and x_2 are the generated and testing RSSI values, the SSD value is $\sqrt{(x_1-x_2)^2/2}$, the MSE value is $(x_1-x_2)^2$. The SSD value is usually smaller than the MSE. In addition, the testing RSSI is easily affected by the environment, and it is not appropriate to directly calculate the difference between the generated and testing RSSI values. If the average of the two RSSI values is adopted, the influence of the environment may be reduced.

The fitness value f_i of the ith particle can be calculated by (11), where sum () is the summation function, std () is the standard deviation function, $\boldsymbol{R}_i^{\mathrm{p}}$ is the generated fingerprint of the ith particle, $R_{i,j}^{\mathrm{p}}$ is the jth RSSI value of the ith particle.

$$f_i = \mathrm{sum}\left(\mathrm{std}\left(\boldsymbol{R}_i^{\mathrm{p}};\boldsymbol{r}\right)\right) = \mathrm{sum}\left(\mathrm{std}\begin{pmatrix} R_{i,1}^{\mathrm{p}} \cdots R_{i,j}^{\mathrm{p}} \cdots R_{i,M}^{\mathrm{p}} \\ r_1 \cdots r_j \cdots r_M \end{pmatrix}\right) \tag{11}$$

Given two vectors $\boldsymbol{R}_i^{\mathrm{p}} = [-81, -13, -63, -28, -96]$ and $\boldsymbol{r} = [-91, -91, -10, -55, -96]$, they denote the generated fingerprint of the particle and the testing fingerprint, respectively. The standard deviation values are $[7.0711, 55.1543, 37.4767, 19.0919, 0]$, and the SSD value is about 118.7939. In this case, the MSE value is about 1944.4, which is a rather big value. The MSE value is much larger than the SSD value. Obviously, the particle is not the target one. The algorithm needs to constantly search for particles with smaller SSD values.

After traversing all particles in one iteration, m fitness values are obtained. For the first iteration, the m fitness values are all treated as local-best ones, and locations of all local-best particles are their own, as shown in (12), where f_i^{pb} denotes the local-best fitness value of the ith particle, $\boldsymbol{l}_i^{\mathrm{pb}}$ denotes the local-best location of the ith particle, $\boldsymbol{l}_i^{\mathrm{p}}$ represents the current location of the ith particle. The global-best fitness value f^{gb} can be obtained by (13) using the minimum function, $\boldsymbol{f}$ is the fitness values vector of the m particles in one iteration. If the index corresponding to the minimum fitness value is denoted as k, then the location of the global-best particle $\boldsymbol{l}^{\mathrm{gb}}$ in the particle swarm is the location of the kth particle $\boldsymbol{l}_k^{\mathrm{p}}$.

$$f_i^{\mathrm{pb}} = f_i, \boldsymbol{l}_i^{\mathrm{pb}} = \boldsymbol{l}_i^{\mathrm{p}}, i = 1, \ldots, m \tag{12}$$

$$f^{\mathrm{gb}} = \min(\boldsymbol{f}) = \min\left(f_1 \cdots f_i \cdots f_m\right), \boldsymbol{l}^{\mathrm{gb}} = \boldsymbol{l}_k^{\mathrm{p}} \tag{13}$$

Otherwise, the newly obtained fitness values should be compared with the best values in history. Once the new smaller fitness value appears in the following iteration, it will replace the corresponding best fitness. As shown in (14), if the fitness value of the ith particle in one iteration is smaller than the local-best fitness value in history, the fitness value will be assigned to the local-best one, and the local-best location of the ith particle should be updated by the current location. At the same time, the global-best particles should also be replaced by the particle with the minimum fitness value.

$$\begin{cases} f_i^{\mathrm{pb}} = f_i, \boldsymbol{l}_i^{\mathrm{pb}} = \boldsymbol{l}_i^{\mathrm{p}}, \text{if } f_i^{\mathrm{pb}} > f_i \\ f^{\mathrm{gb}} = \min(\boldsymbol{f}), \boldsymbol{l}^{\mathrm{gb}} = \boldsymbol{l}_k^{\mathrm{p}}, \text{if } f^{\mathrm{gb}} > \min(\boldsymbol{f}) \end{cases} \tag{14}$$

3.5. Particles update

At one iteration, each particle is changed towards locations of the local-best one and global-best one. Velocities are updated based on velocities of the last iteration, and locations of the particle, the local-best and global-best ones, as shown in (15).

$$\boldsymbol{v}_{i,t+1}^{\mathrm{p}} = w^{\mathrm{p}} \cdot \boldsymbol{v}_{i,t}^{\mathrm{p}} + c_1 \cdot r_1 \cdot \left(\boldsymbol{l}_{i,t}^{\mathrm{pb}} - \boldsymbol{l}_{i,t}^{\mathrm{p}}\right) + c_2 \cdot r_2 \cdot \left(\boldsymbol{l}_t^{\mathrm{gb}} - \boldsymbol{l}_{i,t}^{\mathrm{p}}\right) \tag{15}$$

where $\boldsymbol{v}_{i,t}^{\mathrm{p}}$, $\boldsymbol{l}_{i,t}^{\mathrm{p}}$ and $\boldsymbol{l}_{i,t}^{\mathrm{pb}}$ denote velocities, the current location of the ith and its local-best particle in the tth iteration, respectively; $\boldsymbol{v}_{i,t+1}^{\mathrm{p}}$ indicates velocities of the ith particle in the $(t+1)$th iteration; $\boldsymbol{l}_t^{\mathrm{gb}}$ means the location of the global-best particle in the tth iteration; w^{p} is the inertial weight and ranges from 0.6 to 0.9, c_1 and c_2 are the learning rates and belong to [0, 4], and r_1, r_2 are the separate changing random numbers from 0 to 1.

Then, the location of the ith particle in the $(t+1)$th iteration can be updated using (16).

$$\boldsymbol{l}_{i,t+1}^{\mathrm{p}} = \boldsymbol{l}_{i,t}^{\mathrm{p}} + \boldsymbol{v}_{i,t+1}^{\mathrm{p}} \tag{16}$$

These updated locations and velocities cannot be used for IDW interpolation directly before velocities and locations are constrained. If the location and velocities of a particle are beyond the constraint, the corresponding maximum and minimum values are assigned in (17) and (18). Where $x_{\max}$, $x_{\min}$, $y_{\max}$, and $y_{\min}$ are generated in particles initialization, v_{b} is the boundary velocity assigned in the initialization step.

$$\begin{cases} \boldsymbol{l}^{\mathrm{p}}\left(\boldsymbol{l}^{\mathrm{p}}(:,1) > x_{\max}, 1\right) = x_{\max} \\ \boldsymbol{l}^{\mathrm{p}}\left(\boldsymbol{l}^{\mathrm{p}}(:,1) < x_{\min}, 1\right) = x_{\min} \\ \boldsymbol{l}^{\mathrm{p}}\left(\boldsymbol{l}^{\mathrm{p}}(:,2) > y_{\max}, 2\right) = y_{\max} \\ \boldsymbol{l}^{\mathrm{p}}\left(\boldsymbol{l}^{\mathrm{p}}(:,2) < y_{\min}, 2\right) = y_{\min} \end{cases} \tag{17}$$

$$\begin{cases} \boldsymbol{v}\left(\boldsymbol{v} > v_{\mathrm{b}}\right) = v_{\mathrm{b}} \\ \boldsymbol{v}\left(\boldsymbol{v} < -v_{\mathrm{b}}\right) = -v_{\mathrm{b}} \end{cases} \tag{18}$$

Specific steps of IDWPSOInLoc can be summarized as follows.

Step 1. Particles are initialized based on the selected RPs with minimum distances when a user requests an online real-time localization.

Step 2. In an iteration, all particles obtain interpolated fingerprints based on their locations and the IDW algorithm.

Step 3. Fitness values are calculated on the basis of interpolated fingerprints and the fitness function.

Step 4. The new calculated fitness values would are compared with fitness values of the local-best and global-best particles until the minimum fitness value is attained. If the answer is yes, the particle's location with the global minimum fitness value is treated as the final estimated result, and the algorithm is terminated. Otherwise, the algorithm proceeds to the next step.

Step 5. Update velocities and locations of all particles. Repeat steps from Step 2 to Step 5 until the end of iterations.

3.6. Time complexity analysis

The overall time complexity of IDWPSOInLoc can be obtained by analyzing the time complexity of the five procedures, RPs' selection, initialization, interpolated fingerprint, fitness values calculation, and particles' update. The fingerprint database has N RPs, Each fingerprint contains M RSSI values. There is m particles in the swarm. And assumed that the number of iterations is I.

- KNN selects K RPs from a $M \times N$ matrix, the time complexity of RPs' selection is $\mathrm{O}(M * N)$.
- Locations and velocities (4 values) of the m particles in the entire swarm are initialized, the time complexity of initialization for the m particles is $\mathrm{O}(4 * m)$.
- The interpolated fingerprint (M RSSI values) is generated for each particle at one iteration, the time complexity of the interpolated fingerprint for the m particles is $\mathrm{O}(M * m)$.
- The fitness value is calculated for each particle using interpolated fingerprint at one iteration. The main calculation solves the standard deviation, and its time complexity is quadratic time. Then, the time complexity of fitness values calculation for the m particles is $\mathrm{O}\left(M^2 * m\right)$.
- Locations and velocities are updated for each particle at one iteration, the time complexity of particles' update is $\mathrm{O}(4 * m)$.

The two former procedures are adopted at the beginning, the three latter are explored in each iteration. So the entire time complexity is $\mathrm{O}\left(M * N + m * \left(I * M + I * M^2 + 4 * I + 4\right)\right)$. In many cases, M and N are much larger than I and m, assuming that $I * m$ is approximate to N, the entire time complexity becomes $\mathrm{O}\left(M^2 * N + 2 * M * N\right)$. Then, the final time complexity is $\mathrm{O}\left(M^2 * N\right)$.

Smartphones now integrated special hardware on-chips that can enable them to support the inference process of various machine learning and deep learning methods, such as MobileNet and NCNN. IDWPSOInLoc, with lower complexity, can effectively operate on many brands of smartphones, except for Apple iPhones. This is because Apple devices employed random media access control addresses when performing WiFi scans, which limits the functionality of IDWPSOInLoc on Apple devices.

4. Experiment and performance evaluation

This section includes the experimental description and performance evaluation of IDWPSOInLoc using the fingerprint data collected in the local environment and the public UJIIndoorLoc dataset. WKNN [22], PSO-SVM [52,57], PSO-BP [32], convolutional nerual network (CNN)

[58], and deep nerual network (DNN) [59] are separately introduced into comparisons on different evaluation metrics. The WKNN positioning algorithm selects K nearest RPs and assigns weights to them, and then outputs the weighted mean coordinate as the estimated location. In the PSO-SVM and PSO-BP positioning algorithms, SVM and BP models are established based on fingerprint database, respectively. The PSO algorithm is employed to find the optimal parameters for the SVM and BP models. CNN positioning algorithm is one of the most commonly used deep learning method in the field of indoor positioning and indoor navigation. It firstly transforms RSSI fingerprints into images, and then constructs the mapping between images and coordinate values through convolutional layer, relu layer, fully-connected layer, and regression layer forming a network model. The DNN model has 7 fully connected hidden layers, as well as an input layer and an output layer. The input layer of the DNN is RSSI fingerprints, and the output layer of the DNN is the predicted coordinates. And stacked autoencoders is adopted to reduce the dimensionality of RSSI fingerprints. Once the real-time fingerprint is provided, the optimized SVM and BP models, CNN, and DNN positioning models will generate the positioning results.

The mean absolute error (MAE), cumulative distribution function (CDF), 50%, 75%, and 95% error, and the root mean square error (RMSE) are introduced to evaluate the positioning performance. Furthermore, a computational complexity analysis, focusing on time and memory, is conducted to evaluate the computational performance of different algorithms. The simulation platform is MATLAB 2016a running on a computer with a Windows 10 operating system, a processor of Intel seventh generation 9700k in 3.6 GHz, and random access memory (RAM) in 16 GB.

For the local environment experimental test, the number of particles and iterations are 50 and 150, both c_1 and c_2 are set as 1, w^{p} is 0.8 in the PSO algorithm. They are utilized to optimize the model parameters of SVM and BP. Due to the inherent variability in IDWPSOInLoc's results across multiple tests, ten tests are performed for comprehensive comparisons. The best positioning performance of WKNN are adopted to compare with other methods. The selected SVM model is the ϵ-SVR, and the kernel function is the radial basis function. The maximal and minimal penalty parameters are 100 and 0.01, respectively. The maximal and minimal kernel widths are 1000 and 0.01, respectively. The epsilon deviation is 0.01. The number of input layers in BP is the dimension of features, i.e., M, and the number of hidden layer nodes is the half of the number of RPs, i.e., $N/2$. Stochastic gradient descent with momentum is the solver to use for training the network. The maximum number of epochs for training is set as 100. For each training iteration, the mini batch size is set as 8. The learning rate is 0.001.

For the public UJIIndoorLoc dataset, the number of particles and iterations are also 50 and 150, both c_1 and c_2 are set as 2, w^{p} is 0.9. DNN [59] provided detailed parameters of the DNN model. And positioning results of the DNN model in UIJIndoorLoc public dataset are directly compared with those of IDWPSOInLoc.

4.1. Experimental description and Evaluation metrics

4.1.1. Experimental description of the local environment

The layout of the experimental testbed is illustrated in Fig. 4. It covers an area of 852 square m with a length of 57 m. The signal transmitter, the blue solid dot, and the green solid square represent AP, RP, and testing point (TP), respectively. The main localization scenario is a corridor. Ten APs and 110 RPs are evenly distributed in the experimental scene. AP transmits wireless signals at dual bands, 2.4 GHz and 5 GHz. The interval between two adjacent RPs is about 1.2 m. RSSI samples are collected for 30 s at each RP with a sampling frequency of 1 Hz. These RSSI collections are used to generate reference fingerprints by average processing.

In the experimental testbed, we have deployed 28 TPs. RSSI values are collected at each TP for a duration of 10 s with a consistent sampling frequency of 1 Hz. RSSI collection in one second was simulated as a real-time online measurement. In total, 280 sets of RSSI fingerprints are collected for the purpose of evaluating localization performance. When the collector stands at RP or TP, all RSSI values are acquired by using a self-developed application software on a smartphone. During the process of RSSI collection, pedestrians walk normally through the corridor, and occasionally open doors on either side of the corridor. The collected RSSI values usually fluctuate greatly and unstable, bringing difficulties to the accurate and stable localization. Moreover, the same smartphone is employed to collect RSSI samples at both RPs and TPs without device heterogeneity.

4.1.2. Experimental description of the selected floor in the UJIIndoorLoc dataset

The UJIIndoorLoc dataset includes 3 buildings, the building 0 and building 1 with 4 floors and the building 2 with 5 floors, covering a surface of 108 703 square m. It is one of the typical representatives of large-scale WiFi localization datasets. Many researches validate their algorithms using this dataset and demonstrate positioning results of the whole dataset, such as CNN [42,60–62], DNN [43,59], ELM [39,40] and RNN [63]. The statistics of positioning error using deep learning are shown in Table 1. The best positioning performance of these 8 algorithms is EA-CNN with the MAE of 8.34 m. In [59], MAEs of each building and floor are separately provided. The MAE of the floor 3 in the building 0 is lower than other floor in other buildings, and the value is about 7.1 m. We select fingerprints of the floor 3 in the building 0 to conduct several positioning experiments, shown in Fig. 5. There are 1391 and 85 samples in training and testing datasets, respectively. The number of unique APs in the above two datasets is 201.

4.1.3. Evaluation metrics

MAE is the most commonly used metric to evaluate positioning accuracy. The smaller the value, the higher the positioning accuracy. MAE can be calculated by (19). Where N_{t} is the number of tests, e_i is the absolute error of the ith test, $(\hat{x}_i, \hat{y}_i)$ and (x_i, y_i) are estimated coordinates and corresponding actual coordinates, respectively.

$$\mathrm{MAE} = \frac{1}{N_{\mathrm{t}}} \sum_{i=1}^{N_{\mathrm{t}}} e_i = \frac{1}{N_{\mathrm{t}}} \sum_{i=1}^{N_{\mathrm{t}}} \sqrt{\left(\hat{x}_i - x_i\right)^2 + \left(\hat{y}_i - y_i\right)^2} \tag{19}$$

Assuming that the number of positioning error no more than a specific error e is n_e, the percentage of positioning error under this specific error can be obtained by (20). CDF can reflect the distribution of positioning error. The horizontal axis shows positioning error, and the vertical axis represents corresponding cumulative probabilities in percentages.

$$\Phi(e) = P\left(e_i \le e\right) = \frac{n_e}{N_{\mathrm{t}}} \tag{20}$$

CDF is usually illustrated as a curve. Heights of curves can be compared to determine which algorithm achieves higher positioning accuracy. The higher the curve, the higher the positioning accuracy. And positioning accuracy can be compared by different percentages under a specific error or error corresponding to a certain percentage.

RMSE is the square root of the arithmetic mean of squares error, as shown in (21). It is a measure of the deviation between the observed values and the true values. To some extent, RMSE could reflect the variation between the estimated location and the known one.

$$\mathrm{RMSE} = \sqrt{\frac{1}{N_{\mathrm{t}}} \sum_{i=1}^{N_{\mathrm{t}}} e_i^2} \tag{21}$$

4.2. Performance evaluation using the local environment experimental dataset

4.2.1. The positioning error and runtime of 10 IDWPSOInLoc tests

PSO uses several random functions to avoid local convergence. This operation also makes the results of PSO different each time. Therefore, ten tests are carried out to determine the positioning error and runtime

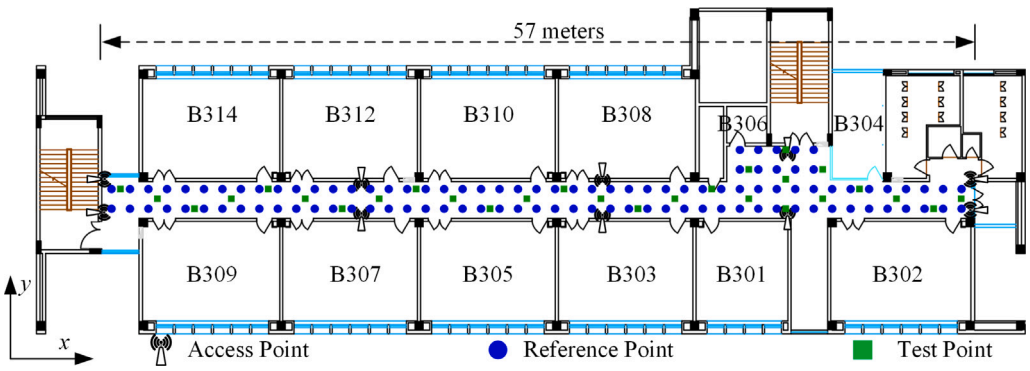


Fig. 4. The layout of the experimental testbed.

Table 1
Statistics of positioning error of 8 algorithms using the UJIIndoorLoc dataset (m).

Error (m)	CNNLoc [60]	C-CNNLoc [61]	CDAE-CNN [62]	HADNN [43]	RNN [63]	HELM [39]	EA-CNN [42]	DNN [59]
MAE	11.78	8.87	12.4	14.93	8.66	8.42	8.34	9.02
RMSE	–	–	–	–	–	12.5	10.72	–

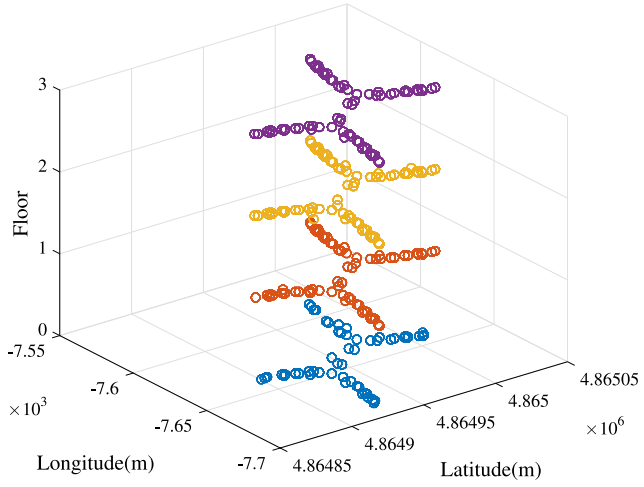


Fig. 5. The layout of building 0 in the UJIIndoorLoc dataset.

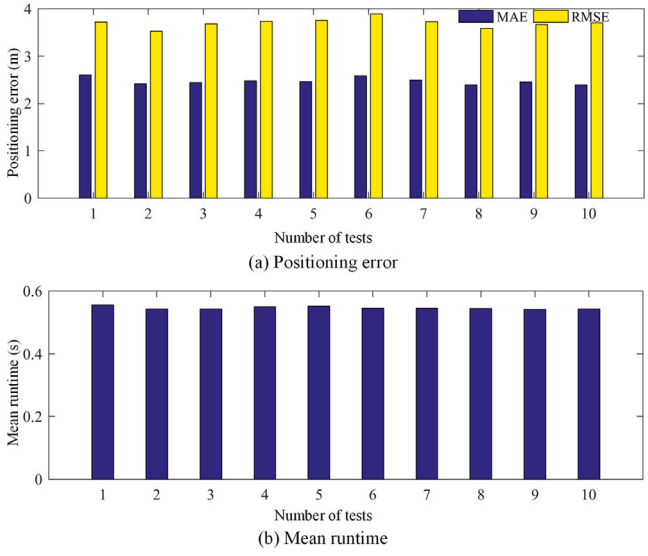


Fig. 6. Statistics of positioning error and mean runtime.

of IDWPSOInLoc. Fig. 6 shows statistics of positioning error and mean runtime of 10 tests. The mean runtime is the arithmetic average time of 280 positioning requests for each test.

For each test, MAE, RMSE, and the mean runtime are less than 2.7 m, 3.9 m, and 0.56 s, respectively. For the 2800 positioning calculations, the MAE is about 2.471 m, the RMSE is 3.700 m, and the mean runtime is about 0.546 s. In the fourth test, the MAE is 2.477 m, the RMSE is 3.737 m, and the mean runtime is 0.549 s. Their positioning error are very approximate. Can we replace the 2800 samples with the fourth test for further performance comparisons? This process requires a statistical rank test.

We assume that the fourth and the other nine tests come from populations with the same continuous distribution, and all observations are mutually independent. And the Kruskal–Wallis test [64] is adopted to evaluate the statistical rank test. It is a nonparametric version of classical one-way ANOVA and an extension of the Wilcoxon rank sum test to two or more groups. It compares the medians of the groups if the samples come from the same distribution. A chi-square statistic replaces the F-statistic used in classical one-way ANOVA, and the p-value measures the significance of the chi-square statistic. p is a scalar value in the range [0,1]. p is the probability of observing a test statistic as extreme as, or more extreme than, the observed value under the null hypothesis. Small values of p cast doubt on the validity of the null hypothesis. In our case, the null hypothesis is that there is no significant difference between the fourth test and the other nine tests, and the significance level is 0.05. The following Fig. 7 illustrates that the fourth test and the other nine tests have the same distribution except for the median positioning error. The returned p value of the Kruskal–Wallis test is 0.9338, larger than the significance level. The null hypothesis is accepted. Therefore, although the results of the fourth test are slightly worse than the total mean value, they are utilized for subsequent performance evaluations.

4.2.2. Accuracy of IDWPSOInLoc and other positioning algorithms

In Fig. 8, black solid line, magenta dashed line, cyan dotted line, blue dotted line and green dash-dot line illustrate CDFs of positioning error of WKNN, PSO-SVM, PSO-BP, CNN and IDWPSOInLoc, respectively. The green dash-dot line is higher than the other three lines. When a line perpendicular to the horizontal axis goes through these CDF lines, IDWPSOInLoc gets a larger percentage with the same positioning error. When a line perpendicular to the vertical axis goes through these CDF lines, IDWPSOInLoc obtains a smaller positioning error with the same percentage. Percentages of positioning error within 1 m of WKNN, PSO-SVM, PSO-BP, CNN and IDWPSOInLoc are

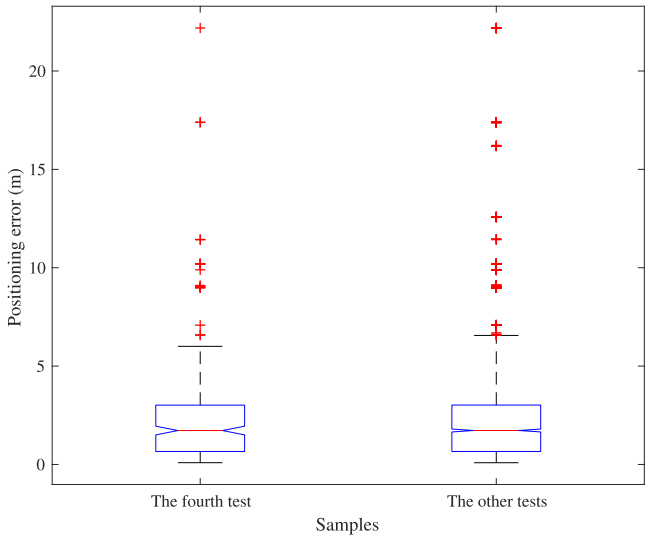


Fig. 7. Distribution of positioning error on different samples.

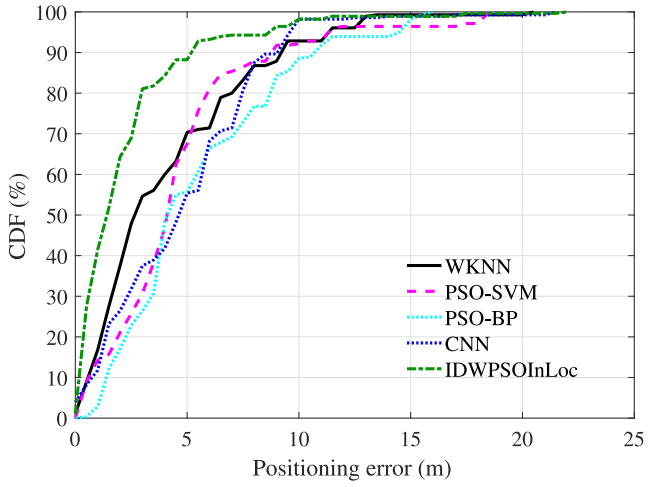


Fig. 8. CDFs of positioning error of 5 algorithms.

Table 2
Statistics of positioning error of 5 algorithms (m).

Algorithms	MAE	50% error	75% error	95% error	RMSE
WKNN	3.952	2.392	5.775	12.621	5.385
PSO-SVM	4.846	4.283	5.645	11.422	6.078
PSO-BP	5.677	4.258	7.816	14.701	6.754
CNN	4.957	4.883	7.663	9.752	5.997
IDWPSOInLoc	2.477	1.724	3.016	8.980	3.737

21.79%, 10.71%, 2.14%, 8.21% and 32.14%, respectively. At the end of lines, the green dash-dot line is longer than the other three lines, and IDWPSOInLoc has a relatively large maximum error.

Table 2 shows the positioning error of five algorithms. With regard to MAE, 50%, 75%, and 95% error, and RMSE, IDWPSOInLoc obtains the reduction rates of 37.32%, 27.93%, 46.57%, 21.38%, 30.60% at least. Although the proposed IDWPSOInLoc behaves better positioning performance than CNN, they have approximate positioning accuracy in y direction. The mean positioning error of IDWPSOInLoc in both x and y directions are 2.350 m and 0.439 m, respectively, while those of CNN are 4.843 m and 0.465 m. Large error in x direction is the main reason why the positioning performance of CNN is worse than that of IDWPSOInLoc.

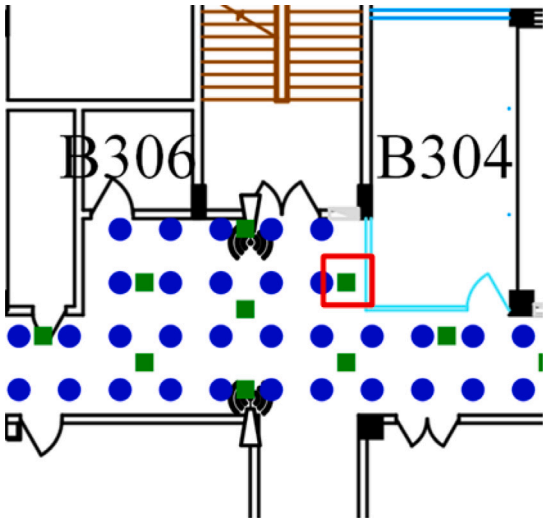


Fig. 9. The particular TP in a red square.

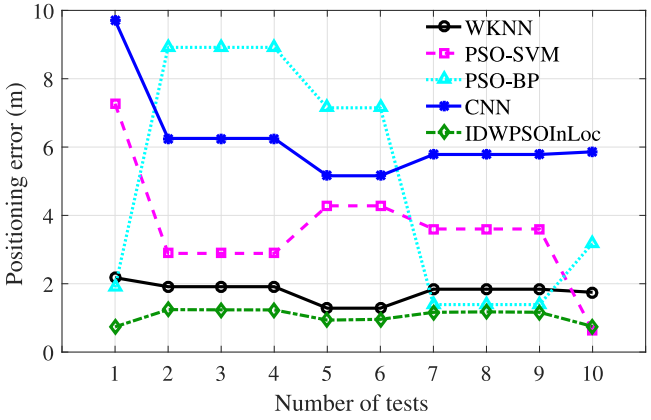


Fig. 10. Positioning error of four algorithms for ten positioning tests at the particular TP.

In the experimental testbed, a particular TP is deployed outside the boundaries of RPs, as shown in Fig. 9. Some RPs are near the particular TP but do not surround it. In other words, the location estimation of the particular TP belongs to extrapolation rather than interpolation for positioning methods using regression models. WKNN and IDWPSOInLoc belong to the category of matching localization methods. Therefore, the positioning error of WKNN and IDWPSOInLoc are smaller than those of PSO-SVM, PSO-BP and CNN, as shown in Fig. 10. Among them, positioning error of IDWPSOInLoc are the smallest and behave with the strongest stability at the particular TP.

4.2.3. *Comparisons of different fitness functions*

Distances between two sets of RSSI fingerprints are diverse, such as Euclidean distance and Hamming distance. To verify the effectiveness of the proposed fitness function, ten positioning tests of IDWPSOInLoc are conducted using fitness functions based on SSD and MSE with both Euclidean and Hamming distances.

Ten MAEs of IDWPSOInLoc based on different fitness functions are displayed in Fig. 11. The blue line with circles, red dotted line with squares, and green dash-dot line with diamonds denote positioning MAEs based on MSE with Hamming distance, MSE with Euclidean distance, and SSD, respectively. IDWPSOInLoc based on MSE-Hamming has the poorest positioning performance, and all the MAEs are larger than 5.5 m. MAEs of IDWPSOInLoc based on MSE-Euclidean are larger

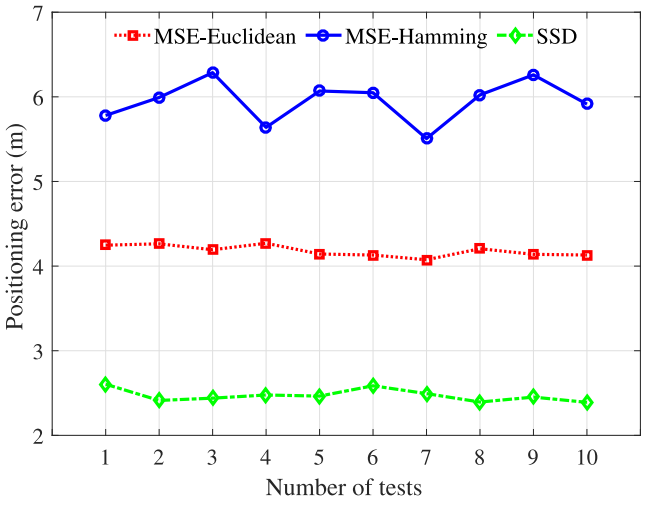


Fig. 11. Positioning MAEs of IDWPSOInLoc based on different fitness functions.

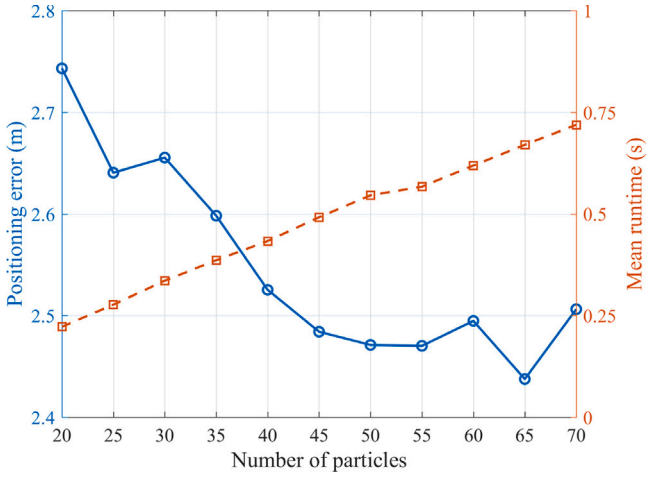


Fig. 12. Positioning error and mean runtime vs. the number of particles.

Table 3
Statistics of mean runtime and allocated memory.

Algorithms	Mean runtime (s)	Allocated memory (kB)
WKNN	0.027	524
PSO-SVM	0.263	229 088
PSO-BP	3.213	342 863 868
IDWPSOInLoc	0.546	3546

than 4 m. Among them, IDWPSOInLoc based on SSD achieves the best positioning performance because all MAEs are less than 2.7 m.

4.2.4. Statistics of the computational complexity of different positioning algorithms

Computational complexity contains time and memory complexity, which can be evaluated by mean runtime and allocated memory of different positioning algorithms. The mean runtime is obtained based on the operating system time. Table 3 shows the statistics of the mean runtime and allocated memory. PSO-BP takes the most runtime and memory, while WKNN consumes the least. WKNN is very simple and easy to implement, and it is widely used in indoor positioning. It is the main reason for WKNN to get the lowest mean runtime and allocated memory. In the positioning experiment, there are 10 dual-band APs and 110 RPs, i.e., M is 20. Based on Section 3.6, the time complexity of the proposed IDWPSOInLoc is 20 times than those of WKNN. And the mean runtime of IDWPSOInLoc in Table 3 is 20 times longer than that of WKNN. These results are consistent with the theory that PSO-BP is much more complicated than WKNN, PSO-SVM, and IDWPSOInLoc. And 96 690 times more memory is required for PSO-BP than IDWPSOInLoc. Although the allocated memory of PSO-SVM is 64.6 times more than that of IDWPSOInLoc, PSO-SVM spends less runtime than IDWPSOInLoc. The reason for fewer runtime of PSO-SVM than that of IDWPSOInLoc is that PSO-SVM adopts the LIBSVM package, whose core programs are written in C/C++ language [65]. LIBSVM dramatically improves the running efficiency of PSO-SVM to some extent, while the matrix-based MATLAB language implements IDWPSOInLoc. Regarding mean runtime and allocated memory, the computational complexity of IDWPSOInLoc is not very high. CNN uses the graphics processing unit to implement localization, while the other algorithms utilizes the central processing unit, the complexity of CNN is not counted for statistics.

4.2.5. Effect of the number of particles on positioning performance in IDWPSOInLoc

The number of particles has a certain effect on the positioning performance of IDWPSOInLoc. Each particle comprises the attribute of location coordinates. Theoretically, the more particles there are, the more likely IDWPSOInLoc will get an optimal location estimation.

The numbers of particles are chosen from 20 to 70 with intervals of five in the experiment. Fig. 12 illustrates the relationship concerning positioning error and mean runtime vs. the number of particles. The blue line with circles and red dotted line with squares display the variation of positioning error and mean runtime, respectively. With the increase of the number of particles, positioning error generally decreases, and on the contrary, the required time gradually increases, whereas positioning error are not strictly gradually decrease with slight fluctuations. When the number varies from 20 to 60, the trend of positioning error decreases from 20 to 40 and becomes stable from 45 to 60, with positioning error less than 2.5 m and mean runtime less than 0.6 s. When the number of particles is set as 65, positioning error is the smallest, 2.438 m, but the mean runtime is 0.670 s. Considering both positioning accuracy and mean runtime, the number of particles is suggested as 50.

4.3. Performance evaluation using the selected UJIIndoorLoc dataset

Twenty tests are conducted based on the proposed IDWPSOInLoc for performance evaluation. PSO-SVM and IDWPSOInLoc are operated using the MATLAB 2016a platform. Up to now, the positioning program of PSO-BP has been running continuously for 15 days, but there is still no positioning result. Therefore, positioning error are not exhibited in Table 4. DNN [59] employed 5 algorithms to compare positioning performance with the DNN model based on the UJIIndoorLoc dataset. Table 4 shows positioning error of RandomTree, SVM, DecisionTree, ExtraTree, KNN, DNN, PSO-SVM and IDWPSOInLoc using the selected UJIIndoorLoc dataset. MAEs of the former 6 algorithms are extracted from an image of [59] using the grabit tool [66].

Fig. 13 shows positioning error statistics of the proposed IDWPSOInLoc for twenty positioning tests. The eighth test acquires the minimum MAE and RMSE, which shown at the last column in Table 4. The maximum MAE and RMSE are separately 6.255 and 8.687 m, located at the twelfth test. MAE and RMSE of the whole twenty tests are 9.633 and 12.056 m, respectively. Compared with the other seven algorithms, IDWPSOInLoc achieves the best positioning performance with the positioning error decreased by 11.9% at least.

267.701 and 0.844 s are separately the mean runtime of PSO-SVM and IDWPSOInLoc using the selected UJIIndoorLoc dataset. In this case,

Table 4
Positioning error of 8 algorithms using the selected UJIIndoorLoc dataset (m).

Error	RandomTree	SVM	DecisionTree	ExtraTree	KNN	DNN	PSO-SVM	IDWPSOInLoc	
								Whole	Minimum
MAE	53	31.2	23.9	18.7	11.2	7.1	9.633	6.017	5.729
RMSE	–	–	–	–	–	–	12.056	8.320	7.945

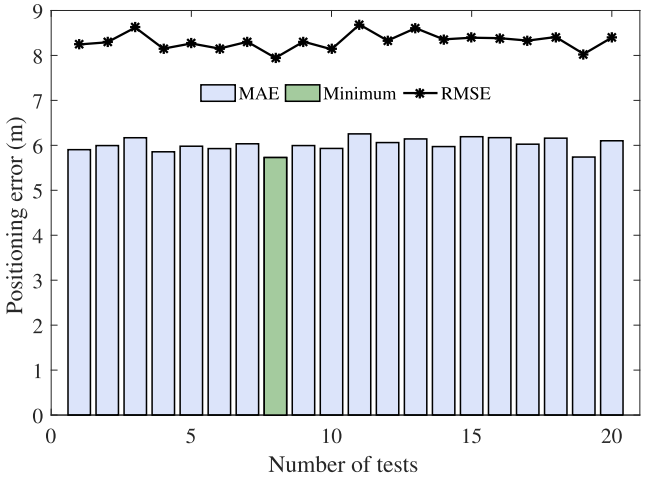


Fig. 13. Positioning error of the proposed IDWPSOInLoc for twenty positioning tests.

M is 201, ten times of that in the local environment experimental test. The mean runtime of PSO-SVM increased by about 1000 times. However, the mean runtime of IDWPSOInLoc increased by only 1.5 times. Above all, IDWPSOInLoc behaves better performance in positioning accuracy and complexity than state-of-the-art algorithms.

5. Discussion

The term *large area* is commonly used in the field of indoor positioning and indoor navigation to describe indoor environmental scenarios. Wikipedia and Google do not provide an official definition and standard specification of it. Despite its frequent use in research papers [67,68], there is little discussion. Moreover, there is currently no convincing unified division to clearly represent what is a large area.

Multi-building-multi-floor buildings are considered as large areas in some studies [68–70], such as teaching buildings, apartments, offices, markets, etc. These buildings trend to cover vast areas with complex interior structures. And these studies focused on WiFi indoor positioning. The main reason is that WiFi devices are already installed in many buildings, reducing the installation workload. Other researches [71–75] declared indoor scenarios with specific area size as large areas, though the coverage sizes varied significantly. Mathisen [71] conducted a comparative evaluation of several indoor positioning techniques at a large university hospital spanning 160,000 square m, utilizing existing WiFi infrastructure. Bae [72] performed geomagnetic positioning experiments in two testbeds, dividing them into medium scale (94 m × 26 m) and large scale (608 m × 50 m). However, such division was primarily for convenience in expression without a solid underlying basis. Ridolfi [73] evaluated the proposed UWB localization system in 250 m cycling and 60 m running tracks, referring to them as two large-scale indoor environments. Chen [74] conducted a dynamic positioning experiment using acoustic method in a large test site measuring 60 m × 30 m. Widmaier [75] carried out an actuarial CSI positioning experiment in indoor environment covering a large area of 80 square m.

Overall, there is no universally definition, standard specification, evaluation index for the term *large area* in the field of indoor positioning and indoor navigation. Different studies have employed diverse criteria to define large areas, leading to variations in coverage sizes and approaches. Determining whether an experimental scene belongs to a large area is often subjective and artificial. Furthermore, the coverage size of an experimental scene is not the sole evaluation index, as it can vary depending on the positioning technologies being utilized. A large area is relative and can differ based on the specific positioning techniques employed.

6. Conclusion

This study proposes an inverse distance weight-assisted particle swarm optimized indoor localization algorithm, a novelty optimized positioning process in the online stage rather than an offline optimization approach. The sum of standard deviation is proposed as the fitness function. Initial locations of particle swarm are formed based on selected reference points. Each particle's fingerprint is generated using an inverse distance weight algorithm and reference points around the particle. The particle swarm optimization algorithm is utilized to find the optimal particle with the minimum fitness, and its corresponding position is taken as the estimated position.

Several experiments were conducted to evaluate the positioning performance of IDWPSOInLoc in terms of positioning accuracy and computational complexity. Compared with state-of-the-art algorithms, IDWPSOInLoc behaves better positioning performance using both the local environment and the selected UJIIndoorLoc datasets.. IDWPSOInLoc also exhibited notably higher positioning accuracy than other methods at a specific testing point, which located outside the boundaries of reference points. Besides, comparisons of fitness functions and the effect of the number of particles on positioning performance are carried out for further evaluation. Experimental results also show that IDWPSOInLoc requires less allocated memory than PSO-SVM and PSO-BP.

However, IDWPSOInLoc still has some limitations. The PSO algorithm utilizes several random functions, which make the global results random, leading to inconsistent positioning results at the same position. In addition, the maximum positioning error of IDWPSOInLoc is larger than those of the other algorithms. Such a significant positioning error is bound to affect the final positioning effect. To address this issue, it may be necessary to combine IDWPSOInLoc with other high-precision positioning methods, such as IMU, UWB, and camera techniques. Fortunately, the complexities of IDWPSOInLoc in time and space are lower, and common smartphones integrate abundant sensors. The hybrid of IDWPSOInLoc and other positioning methods could run on common smartphones for high-precision localization results. In the future, we will focus on improving online optimization algorithms and hybrid localization based on multiple sources.

CRediT authorship contribution statement

Jingxue Bi: Conceptualization, Methodology, Writing – original draft, Writing – review & editing. **Jianhui Wang:** Data curation, Formal analysis. **Hongji Cao:** Investigation, Methodology, Validation. **Guobiao Yao:** Funding acquisition, Software, Supervision. **Yunjia Wang:** Project administration, Resources, Validation. **Zengke Li:** Investigation, Supervision. **Meng Sun:** Visualization, Writing – review & editing. **Hongchao Yang:** Writing – review & editing. **Jie Zhen:** Supervision, Writing – review & editing. **Guoqiang Zheng:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

Acknowledgments

This work was supported by the Youth Innovation Team Project of Higher School in Shandong Province, China [grant number 2023KJ121]; the National Natural Science Foundation of China [grant numbers 42001397, 42304047]; the National Key Research and Development Program of China [grant numbers 2016YFB0502102, 2021YFB3900800]; the Liaoning Provincial Key Laboratory of Geospatial Information Technology, China [grant number LNTUGIT2023-1-3].

References

[1] W. Zhang, K. Yu, W. Wang, X. Li, A self-adaptive AP selection algorithm based on multiobjective optimization for indoor WiFi positioning, IEEE Internet Things J. 8 (3) (2020) 1406–1416.
[2] Y. Yu, R. Chen, L. Chen, X. Zheng, D. Wu, W. Li, Y. Wu, A novel 3-D indoor localization algorithm based on BLE and multiple sensors, IEEE Internet Things J. 8 (11) (2021) 9359–9372.
[3] Z. Chen, H. Zou, J. Yang, H. Jiang, L. Xie, WiFi fingerprinting indoor localization using local feature-based deep LSTM, IEEE Syst. J. 14 (2) (2019) 3001–3010.
[4] H. Zhu, Y. Zhuo, Q. Liu, S. Chang, Perceiving accurate CSI phases with commodity WiFi devices, IEEE Trans. Mob. Comput. 17 (9) (2018) 2155–2165.
[5] H. Cao, Y. Wang, J. Bi, S. Xu, M. Si, H. Qi, Indoor positioning method using WiFi RTT based on LOS identification and range calibration, ISPRS Int. J. Geo-Inf. 9 (11) (2020) 627.
[6] P. Spachos, K.N. Plataniotis, BLE beacons for indoor positioning at an interactive IoT-based smart museum, IEEE Syst. J. 14 (3) (2020) 3483–3493.
[7] A. Lindo, E. Garcia, J. Urena, M. del Carmen Perez, A. Hernandez, Multiband waveform design for an ultrasonic indoor positioning system, IEEE Sens. J. 15 (12) (2015) 7190–7199.
[8] Z. Liu, R. Chen, F. Ye, G. Guo, Z. Li, L. Qian, Improved TOA estimation method for acoustic ranging in a reverberant environment, IEEE Sens. J. 22 (6) (2022) 4844–4852.
[9] B. Li, K. Zhao, E.B. Sandoval, A UWB-based indoor positioning system employing neural networks, J. Geovis. Spat. Anal. 4 (2) (2020) 1–9.
[10] P. Dabove, V. Di Pietra, M. Piras, A.A. Jabbar, S.A. Kazim, Indoor positioning using Ultra-wide band (UWB) technologies: Positioning accuracies and sensors' performances, in: 2018 IEEE/ION Position, Location and Navigation Symposium, PLANS, IEEE, 2018, pp. 175–184.
[11] N. Bai, Y. Tian, Y. Liu, Z. Yuan, Z. Xiao, J. Zhou, A high-precision and low-cost IMU-based indoor pedestrian positioning technique, IEEE Sens. J. 20 (12) (2020) 6716–6726.
[12] Y. Duan, H. Li, S. Wu, K. Zhang, INS error estimation based on an ANFIS and its application in complex and covert surroundings, ISPRS Int. J. Geo-Inf. 10 (6) (2021) 388.
[13] R. Wang, W. Wan, Y. Wang, K. Di, A new RGB-D SLAM method with moving object detection for dynamic indoor scenes, Remote Sens. 11 (10) (2019) 1143.
[14] J.-W. Lee, S.-J. Kim, S.-K. Han, 3D visible light indoor positioning by Bokeh based optical intensity measurement in smartphone camera, IEEE Access 7 (2019) 91399–91406.
[15] M. Sun, Y. Wang, S. Xu, H. Yang, K. Zhang, Indoor geomagnetic positioning using the enhanced genetic algorithm-based extreme learning machine, IEEE Trans. Instrum. Meas. 70 (2021) 1–11.
[16] S.-C. Yeh, W.-H. Hsu, W.-Y. Lin, Y.-F. Wu, Study on an indoor positioning system using Earth's magnetic field, IEEE Trans. Instrum. Meas. 69 (3) (2019) 865–872.
[17] X. Gan, B. Yu, X. Wang, Y. Yang, R. Jia, H. Zhang, C. Sheng, L. Huang, B. Wang, A new array pseudolites technology for high precision indoor positioning, IEEE Access 7 (2019) 153269–153277.
[18] B. Zhou, W. Ma, Q. Li, N. El-Sheimy, Q. Mao, Y. Li, F. Gu, L. Huang, J. Zhu, Crowdsourcing-based indoor mapping using smartphones: A survey, ISPRS J. Photogramm. Remote Sens. 177 (2021) 131–146.
[19] B. Huang, Z. Xu, B. Jia, G. Mao, An online radio map update scheme for WiFi fingerprint-based localization, IEEE Internet Things J. 6 (4) (2019) 6909–6918.
[20] P. Bahl, V.N. Padmanabhan, RADAR: an in-building RF-based user location and tracking system, in: Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies, Vol. 2, 2000, pp. 775–784.
[21] M. Youssef, A. Agrawala, The horus location determination system, Wirel. Netw. 14 (3) (2008) 357–374.
[22] D.B. Ninh, J. He, V.T. Trung, D.P. Huy, An effective random statistical method for indoor positioning system using WiFi fingerprinting, Future Gener. Comput. Syst. 109 (2020) 238–248.
[23] B. Shin, J.H. Lee, T. Lee, H.S. Kim, Enhanced weighted K-nearest neighbor algorithm for indoor Wi-Fi positioning systems, in: 2012 8th International Conference on Computing Technology and Information Management, ICCM 2012, Vol. 2, 2012, pp. 574–577.
[24] J. Bi, Y. Wang, X. Li, H. Qi, H. Cao, S. Xu, An adaptive weighted KNN positioning method based on omnidirectional fingerprint database and twice affinity propagation clustering, Sensors 18 (8) (2018) 2502.
[25] P. Mirowski, P. Whiting, H. Steck, R. Palaniappan, M. MacDonald, D. Hartmann, T.K. Ho, Probability kernel regression for WiFi localisation, J. Locat. Based Serv. 6 (2) (2012) 81–100.
[26] M. Zhou, X. Li, Y. Wang, S. Li, Y. Ding, W. Nie, 6G multisource-information-fusion-based indoor positioning via Gaussian kernel density estimation, IEEE Internet Things J. 8 (20) (2021) 15117–15125.
[27] S. Subedi, J.-Y. Pyun, Lightweight workload fingerprinting localization using affinity propagation clustering and Gaussian process regression, Sensors 18 (12) (2018) 4267.
[28] P. Sadhukhan, K. Dahal, P.K. Das, A novel weighted fusion based efficient clustering for improved Wi-Fi fingerprint indoor positioning, IEEE Trans. Wireless Commun. (2022).
[29] F. Li, M. Liu, Y. Zhang, W. Shen, A two-level WiFi fingerprint-based indoor localization method for dangerous area monitoring, Sensors 19 (19) (2019) 4243.
[30] Y. Rezgui, L. Pei, X. Chen, F. Wen, C. Han, An efficient normalized rank based SVM for room level indoor WiFi localization with diverse devices, Mob. Inf. Syst. 2017 (2017) 6268797.
[31] F.J. Aranda, F. Parralejo, F.J. Alvarez, J.A. Paredes, Performance analysis of fingerprinting indoor positioning methods with BLE, Expert Syst. Appl. 202 (2022) 117095.
[32] N. Li, J. Chen, Y. Yuan, X. Tian, Y. Han, M. Xia, A Wi-Fi indoor localization strategy using particle swarm optimization based artificial neural networks, Int. J. Distrib. Sens. Netw. 12 (3) (2016) 4583147.
[33] C. Wang, Z. Shi, F. Wu, J. Zhang, An RFID indoor positioning system by using particle swarm optimization-based artificial neural network, in: 2016 International Conference on Audio, Language and Image Processing, ICALIP, 2016, pp. 738–742.
[34] B. Sulaiman, E. Natsheh, S. Tarapiah, Towards a better indoor positioning system: A location estimation process using artificial neural networks based on a semi-interpolated database, Pervasive Mob. Comput. 81 (2022) 101548.
[35] S. Kumar, R.M. Hegde, N. Trigoni, Gaussian process regression for fingerprinting based localization, Ad Hoc Netw. 51 (2016) 1–10.
[36] W. Sun, M. Xue, H. Yu, H. Tang, A. Lin, Augmentation of fingerprints for indoor WiFi localization based on Gaussian process regression, IEEE Trans. Veh. Technol. 67 (11) (2018) 10896–10905.
[37] M. Nabati, S.A. Ghorashi, R. Shahbazian, Confidence interval estimation for fingerprint-based indoor localization, Ad Hoc Netw. 134 (2022) 102877.
[38] Y. Tao, L. Zhao, X. Shen, Z. Chen, Q. Zhang, WiFi indoor positioning based on regularized online sequence extreme learning machine, Int. J. Image Data Fusion 11 (4) (2020) 268–286.
[39] A. Alitaleshi, H. Jazayeriy, J. Kazemitabar, Affinity propagation clustering-aided two-label hierarchical extreme learning machine for Wi-Fi fingerprinting-based indoor positioning, J. Ambient Intell. Humaniz. Comput. 13 (6) (2022) 3303–3317.
[40] J. Yan, G. Qi, B. Kang, X. Wu, H. Liu, Extreme learning machine for accurate indoor localization using RSSI fingerprints in multifloor environments, IEEE Internet Things J. 8 (19) (2021) 14623–14637.
[41] M. Nabati, S.A. Ghorashi, A real-time fingerprint-based indoor positioning using deep learning and preceding states, Expert Syst. Appl. 213 (2023) 118889.
[42] A. Alitaleshi, H. Jazayeriy, J. Kazemitabar, EA-CNN: A smart indoor 3D positioning scheme based on Wi-Fi fingerprinting and deep learning, Eng. Appl. Artif. Intell. 117 (2023) 105509.
[43] J. Cha, E. Lim, A hierarchical auxiliary deep neural network architecture for large-scale indoor localization based on Wi-Fi fingerprinting, Appl. Soft Comput. 120 (2022) 108624.
[44] J. Torres-Sospedra, R. Montoliu, A. Martínez-Usó, J.P. Avariento, T.J. Arnau, M. Benedito-Bordonau, J. Huerta, UJIIndoorLoc: A new multi-building and multi-floor database for WLAN fingerprint-based indoor localization problems, in: 2014 International Conference on Indoor Positioning and Indoor Navigation, IPIN, IEEE, 2014, pp. 261–270.
[45] J. Bi, Y. Wang, B. Yu, H. Cao, T. Shi, L. Huang, Supplementary open dataset for WiFi indoor localization based on received signal strength, Satell. Navig. 3 (1) (2022) 1–15.

[46] E. Lohan, J. Torres-Sospedra, P. Richter, H. Leppäkoski, J. Huerta, A. Cramariuc, Crowdsourced WiFi-fingerprinting database and benchmark software for indoor positioning, Zenodo Repos (2017).
[47] A. Abdullah, M. Haris, O.A. Aziz, R.A. Rashid, A.S. Abdullah, UTMInDualSymFi: A dual-band Wi-Fi dataset for fingerprinting positioning in symmetric indoor environments, Data 8 (1) (2023) 14.
[48] J. Torres-Sospedra, R. Montoliu, S. Trilles, O. Belmonte, J. Huerta, Comprehensive analysis of distance and similarity measures for Wi-Fi fingerprinting indoor positioning systems, Expert Syst. Appl. 42 (23) (2015) 9263–9278.
[49] Z. Wu, K. Fu, E. Jedari, S.R. Shuvra, R. Rashidzadeh, M. Saif, A fast and resource efficient method for indoor positioning using received signal strength, IEEE Trans. Veh. Technol. 65 (12) (2016) 9747–9758.
[50] X. Cui, J. Yang, J. Li, C. Wu, Improved genetic algorithm to optimize the Wi-Fi indoor positioning based on artificial neural network, IEEE Access 8 (2020) 74914–74921.
[51] J. Wietrzykowski, M. Nowicki, P. Skrzypczynski, Adopting the FAB-MAP algorithm for indoor localization with WiFi fingerprints, in: International Conference Automation, 2017, pp. 585–594.
[52] X. Liu, W. Wang, Z. Guo, C. Wang, C. Tu, Research on adaptive SVR indoor location based on GA optimization, Wirel. Pers. Commun. 109 (2) (2019) 1095–1120.
[53] M.R. Bonyadi, Z. Michalewicz, Particle swarm optimization for single objective continuous space problems: a review, Evol. Comput. 25 (1) (2017) 1–54.
[54] W. Njima, R. Zayani, I. Ahriz, M. Terre, R. Bouallegue, Beyond stochastic gradient descent for matrix completion based indoor localization, Appl. Sci. 9 (12) (2019) 2414.
[55] S.R. Jondhale, M. Sharma, R. Maheswar, R. Shubair, A. Shelke, Comparison of neural network training functions for RSSI based indoor localization problem in WSN, in: Handbook of Wireless Sensor Networks: Issues and Challenges in Current Scenario's, Springer International Publishing, Cham, 2020, pp. 112–133.
[56] R. Hassan, B. Cohanim, O. De Weck, G. Venter, A comparison of particle swarm optimization and the genetic algorithm, in: 46th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference, AIAA, Austin, Texas, USA, 2005, p. 1897.
[57] J. Bi, M. Zhao, G. Yao, H. Cao, Y. Feng, H. Jiang, D. Chai, PSOSVRPos: WiFi indoor positioning using SVR optimized by PSO, Expert Syst. Appl. (2023) 119778.
[58] N. Agah, B. Evans, X. Meng, H. Xu, A local machine learning approach for fingerprint-based indoor localization, in: SoutheastCon 2023, IEEE, 2023, pp. 240–245.
[59] B. Jia, Z. Zong, B. Huang, T. Baker, A DNN-based WiFi-RSSI indoor localization method in IoT, in: International Conference on Communications and Networking in China, Springer, 2020, pp. 200–211.
[60] X. Song, X. Fan, C. Xiang, Q. Ye, L. Liu, Z. Wang, X. He, N. Yang, G. Fang, A novel convolutional neural network based indoor localization framework with WiFi fingerprinting, IEEE Access 7 (2019) 110698–110709.
[61] Y. Oh, H.-M. Noh, W. Shin, C-CNNLoc: Constrained CNN for robust indoor localization with building boundary, Electron. Lett. 57 (10) (2021) 422–425.
[62] F. Qin, T. Zuo, X. Wang, Ccpos: Wifi fingerprint indoor positioning system based on cdae-cnn, Sensors 21 (4) (2021) 1114.
[63] A.E.A. Elesawi, K.S. Kim, Hierarchical multi-building and multi-floor indoor localization based on recurrent neural networks, in: 2021 Ninth International Symposium on Computing and Networking Workshops, CANDARW, IEEE, 2021, pp. 193–196.
[64] P.E. McKight, J. Najab, Kruskal-wallis test, Corsini Ency. Psychol. (2010) 1.
[65] C.-C. Chang, C.-J. Lin, LIBSVM: A Library for support vector machines, ACM Trans. Intell. Syst. Technol. 2 (3) (2011) 1–27.
[66] J. Doke, Grabit: Extract (pich out) data points off image files, 2024, https://www.mathworks.com/matlabcentral/fileexchange/7173-grabit.
[67] J. Tang, Y. Chen, A. Jaakkola, J. Liu, J. Hyyppä, H. Hyyppä, NAVIS-An UGV indoor positioning system using laser scan matching for large-area real-time applications, Sensors 14 (7) (2014) 11805–11824.
[68] S. Xia, Y. Liu, G. Yuan, M. Zhu, Z. Wang, Indoor fingerprint positioning based on Wi-Fi: An overview, ISPRS Int. J. Geo-Inf. 6 (5) (2017) 135.
[69] M. Ramires, J. Torres-Sospedra, A. Moreira, Accurate and efficient Wi-Fi fingerprinting-based indoor positioning in large areas, in: 2022 IEEE 96th Vehicular Technology Conference, VTC2022-Fall, IEEE, 2022, pp. 1–6.
[70] S. Park, D.H. Kim, C. You, Fi-Vi: Large-area indoor localization scheme combining ML/DL-Based wireless fingerprinting and visual positioning, IEEE Access 10 (2022) 127094–127116.
[71] A. Mathisen, S.K. Sørensen, A. Stisen, H. Blunck, K. Grønbæk, A comparative analysis of indoor WiFi positioning at a large building complex, in: 2016 International Conference on Indoor Positioning and Indoor Navigation, IPIN, IEEE, Alcala de Henares, Spain, 2016, pp. 1–8.
[72] H.J. Bae, L. Choi, Large-scale indoor positioning using geomagnetic field with deep neural networks, in: ICC 2019-2019 IEEE International Conference on Communications, ICC, IEEE, Shanghai, China, 2019, pp. 1–6.
[73] M. Ridolfi, S. Vandermeeren, J. Defraye, H. Steendam, J. Gerlo, D. De Clercq, J. Hoebeke, E. De Poorter, Experimental evaluation of UWB indoor positioning for sport postures, Sensors 18 (1) (2018) 168.
[74] R. Chen, Z. Li, F. Ye, G. Guo, S. Xu, L. Qian, Z. Liu, L. Huang, Precise indoor positioning based on acoustic ranging in smartphone, IEEE Trans. Instrum. Meas. 70 (2021) 1–12.
[75] M. Widmaier, M. Arnold, S. Dorner, S. Cammerer, S. ten Brink, Towards practical indoor positioning based on massive mimo systems, in: 2019 IEEE 90th Vehicular Technology Conference, VTC2019-Fall, IEEE, 2019, pp. 1–6.